

Project Proposal

1. Project Title: SpecTrace-CN – Cloud-Native Microservices Platform for Food Traceability and Compliance

2. Project Sponsor: N/A

3. Project Members

Name	Email
Huang Xiangjia	xiangjia.huang@u.nus.edu
Cai Runchen	runchencai@u.nus.edu
Xu Feiyang	feiyangxu@u.nus.edu
Zhu Wenyu	wenyu.zhu@u.nus.edu
Sun Huajian	shjshjshj@u.nus.edu

4. Overview

Food manufacturers depend on material specifications from many ingredient suppliers. When a supplier changes a material, for example by issuing a new soy-lecithin specification with different components, every manufacturer that uses it must work out which formulas and published labels are affected, re-check allergens and re-approve labels. Today this exchange runs on e-mail and spreadsheets, and each manufacturer rebuilds the same allergen, traceability and label-approval logic, security and audit on its own. Ecosystem growth therefore multiplies cost and produces inconsistent controls.

SpecTrace-CN turns this into a two-sided platform on AWS. A supplier publishes a versioned specification once; the platform matches it to the manufacturers' formulas it affects, tells each manufacturer whether its allergen conclusion changes (NO_ACTION or REVIEW_REQUIRED, with version evidence), and routes the required label work through validation, maker-checker approval and publication. These capabilities are delivered as independently deployable microservices with stable APIs and events, so new consumer applications and new services reuse them instead of rebuilding them.

The project targets four outcomes: (1) a working Producer–Consumer interaction around one business Seed; (2) reusable domain services and engineering assets rather than application-specific code; (3) independent, measured scale-out of the compute-heavy Compliance capability; and (4) a secure Golden Path that measurably reduces the effort to add a service or a consumer.

Starting point. The team re-architects its SWE5006 system, SpecTrace, frozen at main commit a3520e1 (25 Sep 2026; re-checked as main HEAD on 26 Sep 2026): a Java 21 Spring Boot modular monolith with React/TypeScript, Spring JDBC + Flyway + MySQL, Docker Compose and a GitHub Actions pipeline (Testcontainers, Playwright, JaCoCo/Sonar, Trivy, OWASP Dependency-Check). Implemented and tested: versioned supplier, material, specification, product and formula data with release; header-based development identity with database RBAC; label drafts with structured declarations; version-bound derived allergens; and validation runs through REST. Not yet available, and therefore in scope: (a) label review, approval and publication exist only as MySQL stored procedures with embedded RBAC and audit writes, with no API or UI; (b) change-impact analysis exists only as schema tables; (c) there is no organisation model to separate suppliers from manufacturers; (d) many foreign keys and two procedures cross the future service boundaries; and (e) there is no cloud environment.

5. General Architecture

SpecTrace-CN is a cloud-native microservices architecture on Amazon EKS in AWS Singapore (ap-southeast-1). DDD strategic design fixes the service boundaries first; integration, deployment, platform engineering and quality-attribute tactics are then layered around them. Runtime services use AWS managed services where they remove operational toil and standard open-source components where portability matters. The application and open-source platform stack also has local Docker Compose equivalents; AWS-managed edge, database and network resources are isolated to staging deployment configuration.

5.1 Bounded Contexts and Data Ownership

The Core Domain is Compliance Intelligence: turning supplier and formula changes into auditable allergen and label-action decisions. Specification, Formulation and Label Workflow are supporting contexts. Identity (Keycloak), notification and audit use generic platform capabilities.

Service	Role	Owned data	Publishes / consumes
Specification	Supporting; upstream, Producer side	supplier, supplier_material, ingredient vocabulary, specification versions and components	Producer API; publishes SpecificationPublished
Formulation	Supporting; manufacturer-owned	product, formula versions and items; released-specification projection	Formula API; consumes SpecificationPublished; publishes FormulaPublished
Compliance	Core; scale-out target	allergen dictionary, ingredient→allergen mappings, rule sets, validation runs/results, impact findings; projections of released specifications, formulas and published labels	Validation, impact-preview and impact-inbox APIs; publishes ImpactFinding
Label Workflow	Supporting	label versions, declarations, validation records, review tasks, approvals, publications, current-label flag; released-formula projection	Workflow API; consumes FormulaPublished and ImpactFinding; publishes LabelPublished

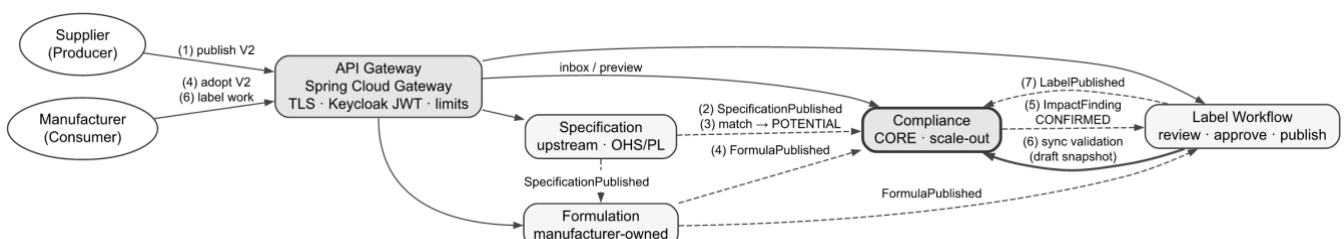


Figure 1. Context map and the two-phase interaction; (n) marks the demo steps in §6.7. Solid = synchronous HTTP; dashed = event via transactional outbox and RabbitMQ.

5.2 Key Architecture Decisions

- **Split the baseline catalog into Specification and Formulation.** Supplier-published specifications are shared network value; formulas are private manufacturer adoption decisions with a different owner, lifecycle and visibility.
- **Specification owns the ingredient vocabulary; Compliance owns allergen knowledge about ingredients. Every dependency therefore points one way, downstream from Specification. An ingredient that Compliance has not mapped yet is reported as UNRESOLVED and yields REVIEW_REQUIRED, never a silent pass (target adaptation of BR-03).**
- **Keep label draft, review, approval and publication in one service.** Maker-checker and replacing the current published label need immediate consistency. Dropping the baseline pointer `product.current_published_label_version_id` in favour of Label Workflow's own current-label flag keeps publication in one local transaction instead of a Saga.
- **Combine allergen derivation, rule evaluation and impact analysis in Compliance: one ubiquitous language and one decision model.** Released versions are immutable (BR-01), so replicas may retain existing version entries in memory; newly published versions are added through events without mutating earlier entries. Compliance is therefore stateless between requests, compute-bound and horizontally scalable.
- **Replace every cross-context foreign key with an ID reference checked against an event-fed projection.** `user_account` becomes the OIDC subject plus an organisation claim, `data_provenance` becomes service-local, and product, formula and rule-set references become plain IDs. No service reads another service's schema.
- **Re-implement the label stored procedures (submit, decision, publish) as service-local transactions in Label Workflow.** Their RBAC joins become token-based checks, and the BR-05 check (an exact-version PASSED validation before review) reads Label Workflow's own validation records instead of `validation_run`. The monolith's existing integration tests are the equivalence oracle.

5.3 Integration, Runtime and Platform Engineering

- **Events.** `SpecificationPublished`, `FormulaPublished`, `ImpactFinding` and `LabelPublished` use versioned JSON Schema as the Published Language. Each service writes business state, local audit and an outbox row in one transaction; a relay publishes them with publisher confirms to a RabbitMQ topic exchange, and each consumer owns a durable quorum queue with a delivery limit and a dead-letter queue (at-least-once). Consumers deduplicate by event ID and discard stale aggregate versions. Every event carries organisation and correlation IDs.
- **One synchronous business call: Label Workflow → Compliance validation.** The request carries the draft snapshot (label version, declarations, formula version, jurisdiction, rule-set version), so Compliance never calls back upstream. An idempotency key makes one bounded retry safe; a 2 s timeout and a circuit breaker make the call fail closed. Label Workflow stores the returned run ID and outcome with the draft, so the later BR-05 check is local.
- **Edge and identity.** An Application Load Balancer (AWS Load Balancer Controller, ACM certificate) terminates TLS in front of Spring Cloud Gateway, the only north-south entry: Keycloak JWT validation, routing, protective request-rate and payload limits, and correlation IDs. Services are ClusterIP-only behind default-deny NetworkPolicies. The React app signs in through Keycloak (authorisation code + PKCE). Each service re-validates the token and enforces roles and organisation ownership; the validation call forwards the user's token so the real actor is authorised and audited. East-west traffic uses Kubernetes service DNS.
- **Golden Path.** A lightweight Spring Boot starter (token and organisation context, correlation ID, error envelope, outbox and idempotent-consumer support, health/readiness, structured logs, metrics), a reusable GitHub Actions workflow and a Helm chart template. It contains no domain entities, DTOs or business rules.

6. Scope of Work

6.1 Platform Business Model

Platform element	SpecTrace-CN MVP
Seed	Versioned supplier material specification: created by suppliers, consumed by manufacturers. The codebase, rule engine and cloud runtime are not Seeds.
Producers	Ingredient suppliers (supplier organisations) that publish specification versions.
Consumers	Manufacturers' formulation, compliance and label teams (manufacturer organisations).
Core interaction	Supplier publishes V2 → platform matches it to formulas that use earlier versions and previews the allergen effect → manufacturer adopts V2 in a new formula → confirmed impact finding → label review, approval and publication.
Magnet	Suppliers publish once instead of answering each customer by e-mail; manufacturers receive impact evidence without asking. Cross-side network effect: more suppliers make the platform more useful to manufacturers, and more manufacturers make publishing more worthwhile for suppliers. A manufacturer that later publishes semi-finished materials also becomes a Producer.
Toolbox	Supplier publication API and client; impact-preview, impact-inbox and validation APIs; event subscriptions; React reference consumer; OpenAPI documents and event schemas.
Matchmaker	Compliance first identifies relevant formulas by material usage, then distinguishes their actual allergen/label effect: REVIEW_REQUIRED only when required allergens are missing or unresolved; otherwise NO_ACTION, with version and provenance evidence.

The MVP intentionally starts with one Seed; regulatory rule packs and other compliance attributes are possible future Seeds and are out of scope.

6.2 Two-Phase Change-Impact Flow (Core Domain)

- **Phase 1 – Match.** On `SpecificationPublished`, Compliance finds released formulas that reference an earlier version of the material, derives the allergen set each would have with the new version, compares it with the current published label and records a POTENTIAL finding (NO_ACTION or REVIEW_REQUIRED) in the manufacturer's impact inbox. A supplier publication never changes a formula (BR-02).
- **Phase 2 – Confirm.** When the manufacturer releases a formula that adopts V2 (`FormulaPublished`), Compliance evaluates it against the current published label and emits a CONFIRMED `ImpactFinding`; Label Workflow opens a review task for REVIEW_REQUIRED.
- **Why Compliance scales independently.** One specification change fans out to every formula of every manufacturer that uses the material, and impact previews and validations follow each change. Compliance load grows with manufacturers × formulas, whereas specification and label traffic grows only with human edits.

6.3 Microservices, Data and Team Responsibilities

The team extracts where the baseline has working logic: catalog → Specification + Formulation; allergen + validation → Compliance (validation already reaches label and formula data through LabelSnapshotPort and FormulaCompositionPort, which become the validation request payload and the formula projection); label + workflow → Label Workflow. New work is the review/approval/publication APIs and screens, the two-phase impact analysis, the organisation model, outbox and events, and the stored-procedure rewrite. Each service keeps Spring JDBC + Flyway and owns one schema with its own credentials on one Amazon RDS for MySQL Multi-AZ instance (primary + synchronous standby) in private subnets; database grants block cross-schema reads.

Organisation model. Every user belongs to one SUPPLIER or MANUFACTURER organisation, carried with the user's roles as token claims. Suppliers can change only their own materials; released specifications are readable by all manufacturers; formulas, labels and findings are visible only to the owning manufacturer.

Member	Primary responsibility
Huang Xiangjia	Specification + Formulation extraction; organisation and ownership model; outbox and event envelope in the starter; SpecificationPublished and FormulaPublished.
Cai Runchen	Compliance extraction; projections; two-phase impact analysis; validation and impact APIs; load and failover experiments; pairs with Sun on the EKS node groups and the Helm chart template in Gate 1.
Xu Feiyang	React reference consumer: Keycloak sign-in, gateway APIs, impact inbox, review/approval/publication screens. Should: partner client and Notification service.
Zhu Wenyu	Label Workflow extraction; stored-procedure rewrite and workflow APIs; Keycloak realm, clients and claim mappers; token/organisation checks in the starter; negative authorisation tests.
Sun Huajian	AWS staging as code with one-command create and destroy (Terraform + Helm: VPC, EKS, RDS, load balancer controller, RabbitMQ and Keycloak operators, Grafana Alloy); per-service CI/CD with GitHub OIDC; evidence bound to commit SHA and image digest.

6.4 Scalability, Availability and Capacity Evidence

- **Workload.** The impact-preview API (evaluate a specification version against a manufacturer's released formulas) reads local projections and in-memory immutable versions, so it exercises Compliance compute rather than database writes; validation writes are measured separately. The dataset is scaled synthetically from the baseline USDA fixtures to about 5,000 released formula versions across 20 manufacturers.
- **Method.** Every Compliance replica has the same fixed limit (1 vCPU, 2 GiB) on a compute-optimized, non-burstable node group (c6i.large), so burst credits and shared-vCPU noise cannot distort the comparison. k6 generates an open arrival-rate load from a separate EC2 host in the same region and steps the request rate until the SLO breaks. Runs: fixed 1 replica, fixed 3 replicas, then HPA autoscaling (1–4 replicas, 60% CPU target) with Cluster Autoscaler growing the node group from 1 to 4 nodes. Metrics: sustainable throughput, p50/p95 latency, error rate, replica CPU, memory and count, plus MySQL CPU and connections to locate the bottleneck.
- **Pre-declared targets (SLO: p95 ≤ 2 s and errors < 1%).** (1) The maximum sustainable rate of 3 replicas is at least 2× that of 1 replica. (2) Should: after a step to a rate one replica cannot sustain, autoscaling restores the SLO within 5 minutes. Targets are fixed before measurement and never relaxed; misses are reported with bottleneck analysis. They describe the test envelope, not production capacity.
- **Failover. At about 50% of the measured 3-replica capacity, one Compliance pod is killed. Targets: error rate over the run < 1%, no continuous error window longer than 10 s, p95 back within the SLO within 60 s, replacement pod ready within 3 minutes. Tactics: readiness probes, replicas spread across nodes, load balancing, RDS Multi-AZ standby failover, RabbitMQ quorum queues with DLQ, timeouts and circuit breaker. Nodes, RabbitMQ and the database standby are spread across two Availability Zones, but the experiment kills only a pod, so the project claims pod and node failure resilience; zonal failure is designed for but not tested, and regional HA is out of scope.**

6.5 Platform Engineering and Reuse Evaluation

The React application becomes the reference Consumer and uses only public APIs through the gateway. Two measurements show the benefit of building on the platform. (1) Should: a lightweight partner client reuses the Specification, impact and validation contracts without database access or domain-service changes; the team records onboarding steps, configuration and time to the first successful end-to-end call. (2) Should: a Notification service (e-mail/webhook for REVIEW_REQUIRED findings) is created from the starter, pipeline and Helm template; time and manual steps from creation to staging are compared with those of the first extracted service.

6.6 DevSecOps, Security and Observability

- **Delivery.** A reusable, path-filtered GitHub Actions workflow per service runs unit, Testcontainers integration and contract tests, Sonar, Trivy and OWASP Dependency-Check, pushes an immutable image tagged by commit SHA to GitHub Container Registry, deploys it to EKS with Helm while a staging window is open and runs smoke tests; on every pull request it also deploys the Helm charts to a throw-away kind cluster. The pipeline assumes a scoped IAM role through GitHub OIDC, so no long-lived cloud keys are stored. Blocking checks never use continue-on-error.
- **Security.** Keycloak OIDC; RBAC and organisation ownership checks in every service; TLS at the load balancer; credentials injected by the pipeline from GitHub environment secrets into Kubernetes Secrets, never committed; least privilege through one MySQL user and one RabbitMQ user per service, default-deny NetworkPolicies, security groups, a private-subnet database and scoped IAM roles; negative tests for 401, 403 and cross-organisation access. The legacy X-Auth-Provider/X-External-Subject headers remain only in the local profile.
- **Observability.** Structured logs and metrics, with correlation IDs propagated through HTTP and event envelopes, are shipped by Grafana Alloy to Grafana Cloud (free tier: Prometheus metrics and Loki logs), with dashboards for the scale and failover runs. The demo reconstructs the audit trail by correlation ID with a Loki query. Could: distributed tracing with OpenTelemetry and Grafana Tempo, and a central audit read model.

6.7 Demonstration, Scope Tiers and Boundary

Final demo (numbers match Figure 1): (1) a supplier publishes soy-lecithin Specification V2; (2) SpecificationPublished propagates; (3) Compliance matches formulas that use V1 and shows POTENTIAL findings in the manufacturer's inbox; (4) the manufacturer releases a formula adopting V2; (5) a CONFIRMED REVIEW_REQUIRED finding opens a review task; (6) Label Workflow runs draft, synchronous validation, submission, maker-checker approval and atomic publication; (7) LabelPublished updates the Compliance

projection. The demo then reconstructs the audit trail, shows Compliance scale-out and failover on the dashboard, and delivers a change through the pipeline.

Tier	Scope
Must	Four services independently deployed on EKS with their own schemas; organisation model; Keycloak and Spring Cloud Gateway; outbox with RabbitMQ for the four events; two-phase impact analysis; review/approval/publication APIs and screens; 1- vs 3-replica load test; one failover test; per-service CI/CD to staging; logs, metrics and correlation IDs.
Should (schedule buffer)	Autoscaling run; partner-client onboarding measurement; Notification service via the Golden Path.
Could	Distributed tracing; central audit read model; event-burst test with queue-depth autoscaling; consumer-driven contract tests for every event.

Should items start only after all Must gates pass and are the first to be cut if the schedule slips. The project demonstrates a configured food-label compliance scenario and a platform-engineering method; it does not provide legal certification, multi-region HA, zero-downtime production migration or a general-purpose regulatory rules marketplace. The monolith is kept only as a regression oracle during extraction.

7. Effort Estimates

Total effort is about 50 man-days (about 10 per member). The official 8–20 Oct Project Conduct window is run as one two-week implementation cycle with three internal gates. Before Proposal Review the team does only reversible work: baseline analysis, architecture, contracts, and a throw-away Terraform spike of the staging environment. Staging in AWS ap-southeast-1 runs only on demand: Terraform creates it at the close of Gates 1 and 2, for the Gate 3 experiments and for the presentation, and destroys it after each window (about 7 running days in total); daily development uses the local Docker Compose stack. At current list prices a running day costs about USD 14–16 (EKS control plane, three general and one compute-optimized node, RDS Multi-AZ, load balancer, NAT gateway and storage), about USD 110–140 for the project. Staging runs in one team member's AWS account, whose new-account Free Tier credits (up to USD 200) are expected to cover it; that account pays any remainder.

Period	Stage	Main work and exit criteria
22 Sep – 7 Oct	Proposal & architecture	Baseline frozen at a3520e1; context map; OpenAPI and event schemas v1; test plan; Terraform spike.
8 – 11 Oct	Gate 1: platform skeleton	Starter; Spring Cloud Gateway + Keycloak; EKS, RDS and RabbitMQ staging created at gate close; CI deploys a skeleton of every service.
12 – 16 Oct	Gate 2: services	Four services on their own schemas; organisation model; stored procedures rewritten; review/approval/publication APIs; events flowing; equivalence tests pass.
17 – 20 Oct	Gate 3: impact & evidence	Two-phase impact end to end on staging; load and failover runs; Should items if Gates 1–2 are green; progress report.
21 – 26 Oct	Presentation freeze	Regression, final measured runs, demo rehearsal and presentation.
27 Oct – 16 Nov	Final report	Presentation feedback; measured results, decisions and limitations.

WBS task	Tier	Man-days
Baseline and domain analysis, boundaries, contracts, staging spike	Must	3
Service starter, Spring Cloud Gateway, Keycloak and organisation claims	Must	5
Service extraction, schema split, organisation model	Must	8
Stored-procedure rewrite, equivalence tests, review/approval/publication APIs and screens	Must	7
Outbox, RabbitMQ, four events, projections, idempotent consumers	Must	5
Two-phase impact analysis (core domain)	Must	4
AWS staging as code (Terraform, Helm, operators), on-demand create/destroy and observability baseline	Must	7
Per-service CI/CD and security gates	Must	3
Load and failover experiments (dataset, k6 scripts, runs)	Must	3
End-to-end demo, presentation and final report	Must	2
Autoscaling, partner client, Notification via Golden Path	Should	3
Total		50 (Must 47 + Should 3)